

Sample Implementation for Design Patterns

The sample design patterns are implemented in Java to incorporate the Strategy, Observer, and Factory Method design patterns in the context of an online shopping system.

Strategy Pattern We will start by implementing the Strategy pattern for managing discount strategies:

```

1  // Strategy interface
2  public interface DiscountStrategy {
3      double applyDiscount(double amount);
4  }
5
6  // Concrete discount strategies
7  class FlatDiscountStrategy implements DiscountStrategy {
8      private double discountPercentage;
9
10     public FlatDiscountStrategy(double discountPercentage) {
11         this.discountPercentage = discountPercentage;
12     }
13
14     public double applyDiscount(double amount) {
15         return amount - (amount * discountPercentage / 100);
16     }
17 }
18
19 class TieredDiscountStrategy implements DiscountStrategy {
20     private double threshold;
21     private double discountPercentage;
22
23     public TieredDiscountStrategy(double threshold, double discountPercentage) {
24         this.threshold = threshold;
25         this.discountPercentage = discountPercentage;
26     }
27
28     public double applyDiscount(double amount) {
29         return amount >= threshold ? amount - (amount * discountPercentage / 100) : amount;
30     }
31 }

```

↗ behavior

Observer Pattern Next, we will implement the Observer pattern for real-time product availability monitoring

一个目标对象，一堆观察者
(商品库存监控)

```

1  import java.util.ArrayList;
2  import java.util.List;
3
4  // Subject interface
5  interface ProductSubject {
6      void attach(ProductObserver observer);
7      void detach(ProductObserver observer);
8      void notifyObservers(String message);
9  }
10
11 // Observer interface
12 interface ProductObserver {
13     void update(String message);
14 }
15
16 // Concrete product subject
17 class ProductInventory implements ProductSubject {
18     private List<ProductObserver> observers = new ArrayList<>();
19
20     public void attach(ProductObserver observer) {
21         observers.add(observer);
22     }
23
24     public void detach(ProductObserver observer) {
25         observers.remove(observer);
26     }
27
28     public void notifyObservers(String message) {
29         for (ProductObserver observer : observers) {
30             observer.update(message);
31         }
32     }
33
34     // Method to update product availability
35     public void updateAvailability(String message) {
36         notifyObservers(message);
37     }
38 }
39
40 // Concrete product observer
41 class CustomerNotification implements ProductObserver {
42     private String customerName;
43
44     public CustomerNotification(String customerName) {
45         this.customerName = customerName;
46     }
47
48     public void update(String message) {
49         System.out.println("Dear " + customerName + ", " + message);
50     }
51 }

```

Handwritten notes and annotations:

- A blue bracket groups the `ProductSubject` and `ProductObserver` interfaces.
- A blue arrow points from the comment `// Concrete product subject` to the handwritten text `商品库存` (Product Inventory).
- The comment `// Concrete product observer` is circled in blue.

Factory Method Pattern Now, let's implement the Factory Method pattern for creating instances of different product categories:

```

1  abstract class Product {
2      private String name;
3      private double price;
4      private DiscountStrategy discountStrategy;
5
6      abstract void displayInfo();
7
8      public Product(String name, double price, DiscountStrategy discountStrategy){
9          this.name = name;
10         this.price = price;
11         this.discountStrategy = discountStrategy;
12     }
13
14     public double getPriceAfterDiscount() {
15         return discountStrategy.applyDiscount(price);
16     }
17
18     public String getName() {
19         return name;
20     }
21 }
22
23 // Concrete product types
24 class ApparelProduct extends Product {
25     public ApparelProduct(String name, double price, DiscountStrategy discountStrategy) {
26         super(name, price, discountStrategy);
27         //TODO Auto-generated constructor stub
28     }
29
30     public void displayInfo() {
31         System.out.println(x:"This is an apparel product.");
32     }
33 }

```

```
34
35 // Concrete product types
36 class ElectronicsProduct extends Product {
37     public ElectronicsProduct(String name, double price, DiscountStrategy discountStrategy) {
38         super(name, price, discountStrategy);
39         //TODO Auto-generated constructor stub
40     }
41
42     public void displayInfo() {
43         System.out.println(x:"This is an electronics product.");
44     }
45 }
46
47 // Product factory
48 class ProductFactory {
49     public Product createProduct(String type) {
50         if (type.equalsIgnoreCase(anotherString:"electronics")) {
51             return new ElectronicsProduct(type, price:0, discountStrategy:null);
52         } else if (type.equalsIgnoreCase(anotherString:"apparel")) {
53             return new ApparelProduct(type, price:0, discountStrategy:null);
54         } else {
55             return null;
56         }
57     }
58 }
```